

Proje Erişim ve Değerlendirme Rehberi

TelcoX CRM — Mikroservis Tabanlı Telekom CRM Platformu

Bu ilk sayfa **rapordan bağımsızdır** ve değerlendirme için hazırlanmıştır. Projede yapılan çalışmaların **ne olduğunu ve nerede bulunduğunu** tek bakışta gösterir; her satırdaki bağlantı doğrudan ilgili dosya/klasöre gider. Raporun kendisi bir sonraki sayfada, kapak ile başlar.

Projede neyin nerede olduğu

Bölüm	Ne içerir	Konum
Kaynak kod (repo)	Tüm proje, 12 servis, arayüz, dökümanlar	github.com/ayminacakir/turkcell-telco-crm-microservices
README	Proje özeti, kurulum, mimari, ekip	README.md
Proje raporu	Bu belge (docx + pdf)	docs/PROJE_RAPORU.docx
Sunum dokümanı	Jüri sunumu (detaylı)	docs/SUNUM.md
Kabul testleri	3 senaryo — komut + beklenen sonuç	docs/KABUL_TESTLERI.md
Gereksinimler	33 fonksiyonel gereksinim (FR)	docs/PROJE_GEREKSINIMLERI.md
Event sözleşmeleri	Kafka event şemaları	docs/event-contracts.md
Web arayüzü	Portal + operasyon konsolu	gateway_server/.../static
CI / CD	GitHub Actions build + test	.github/workflows/ci.yml
Kubernetes + HPA	Ölçekleme örneği	k8s/
Scriptler	run-all · stop · status · seed	scripts/

Mikroservisler (kaynak klasörleri)

Geliştirici	Servis	Kaynak klasör
Aymina Çakır	customer-service	customer-service/
	order-service	order-service/
	subscription-service	subscription-service/
Mervenur Küçükkara	billing-service	billing-service/
	payment-service	payment-service/
	usage-service	usage-service/
Nasrulla Emin	product-catalog-service	product-catalog-service/
	notification-service	notification-service/

Geliştirici	Servis	Kaynak klasör
	ticket-service	ticket-service/

Hızlı çalıştırma

```
cd docker && docker compose up -d && cd .. # altyapı: Kafka, 9x PostgreSQL, Redis, Keycloak
./scripts/run-all.sh # 12 servis (status.sh -> 12/12)
./scripts/seed-demo-data.sh # demo veri
# Tarayıcı: http://localhost:8080/TelcoX.html · ops / elif.aydin · sifre: telcox123
```

Port haritası: gateway 8080 · eureka 8761 · config 8888 · keycloak 8085 · customer 9002 · product-catalog 9003 · order 9004 · subscription 9005 · usage 9006 · billing 9007 · payment 9008 · notification 9009 · ticket 9010

TURKCELL

Geleceęi Yazanlar Bootcamp · 2026

TelcoX CRM

Mikroservis Tabanlı Telekom CRM Platformu

Proje Teslim Raporu

HAZIRLAYANLAR

Aymina akır

Mervenur Küçükkara

Nasrulla Emin

12 Mikroservis

Event-Driven

Database-per-Service

JWT / Keycloak

Apache Kafka

github.com/ayminacakir/turkcell-telco-crm-microservices

İçindekiler

1. Yönetici Özeti
2. Problem, Vizyon ve Hedef
3. Tasarım Felsefesi — Neden Mikroservis?
4. Sistem Mimarisi
5. Teknoloji Yığını
6. Mikroservisler — Servis Servis Derinlik
7. Kilit Mimari Desenler
8. Uçtan Uca İş Akışları (3 Senaryo)
9. Güvenlik Mimarisi
10. Web Arayüzü — TelcoX
11. Kalite: Test, CI ve Kabul Kanıtları
12. Karşılaştığımız Gerçek Problemler ve Çözümleri
13. DevOps ve Ölçeklenebilirlik
14. Beklenti vs. Teslim
15. Ekip ve Katkıları
16. Sonuç ve Gelecek Çalışma
17. Hızlı Referans

1. Yönetici Özeti

TelcoX, hayali bir GSM operatörünün **müşteri yaşam döngüsünü uçtan uca** dijitalleştiren bir mikroservis platformudur. Bir abonenin **kaydından (KYC) → siparişe → ödemeye → abonelik aktivasyonuna → kullanım (CDR) takibine → faturaya → destek taleplerine** kadar tüm süreç; 9 iş servisi ve 3 altyapı servisi üzerinde, **event-driven** ve **database-per-service** prensipleriyle akar.

Tek cümlede farkımız: Ödev “çalışan bir MVP” isterken biz **production-grade** bir sistem kurduk — gerçek web arayüzü, uçtan uca güvenlik, dağıtık dayanıklılık desenleri (outbox, saga, idempotency, circuit breaker), CI ve Kubernetes ölçekleme örneğiyle.

Metrik	Değer
Mikroservis	12 (9 iş servisi + gateway + Eureka + config)
Ayrı veritabanı	9 (database-per-service — servisler arası FK yok)
Kafka event tipi	12+ (order.created, payment.completed, subscription.activated, quota.*, invoice.generated, ticket.*)
Fonksiyonel gereksinim	33 / 33 tamamlandı
Kabul senaryosu	3 / 3 uçtan uca doğrulandı
JWT korumalı servis	Gateway + 9 servisin tamamı
Ekip	3 geliştirici · kişi başı 3 servis + ortak altyapı

2. Problem, Vizyon ve Hedef

Problem: Operatör “TelcoX”, mevcut **monolit** CRM’ini parça parça mikroservislere taşımak istiyor. Monolit; tek DB, tek deploy, tek hata noktası demek — bir modül çökünce tüm sistem düşüyor, bir ekip diğerini bekliyor, ölçekleme “hepsi ya da hiçbiri”.

Vizyon: Abonenin tüm temas noktalarını (sales, service, billing, support) **bağımsız ölçeklenebilen, bağımsız deploy edilebilen** servislere ayırmak; servisler arası bağı event ile gevşetmek; her domaini kendi verisinin sahibi yapmak.

Bootcamp hedefleri (hepsi karşılandı): DDD ile bounded context, Spring Boot 3 production servisleri, Spring Cloud topolojisi, Kafka event-driven entegrasyon, REST + OpenAPI sözleşmeleri, database-per-service, Redis cache/idempotency, Docker + Kubernetes, JWT güvenlik, Resilience4j, CI/CD.

3. Tasarım Felsefesi — Neden Mikroservis?

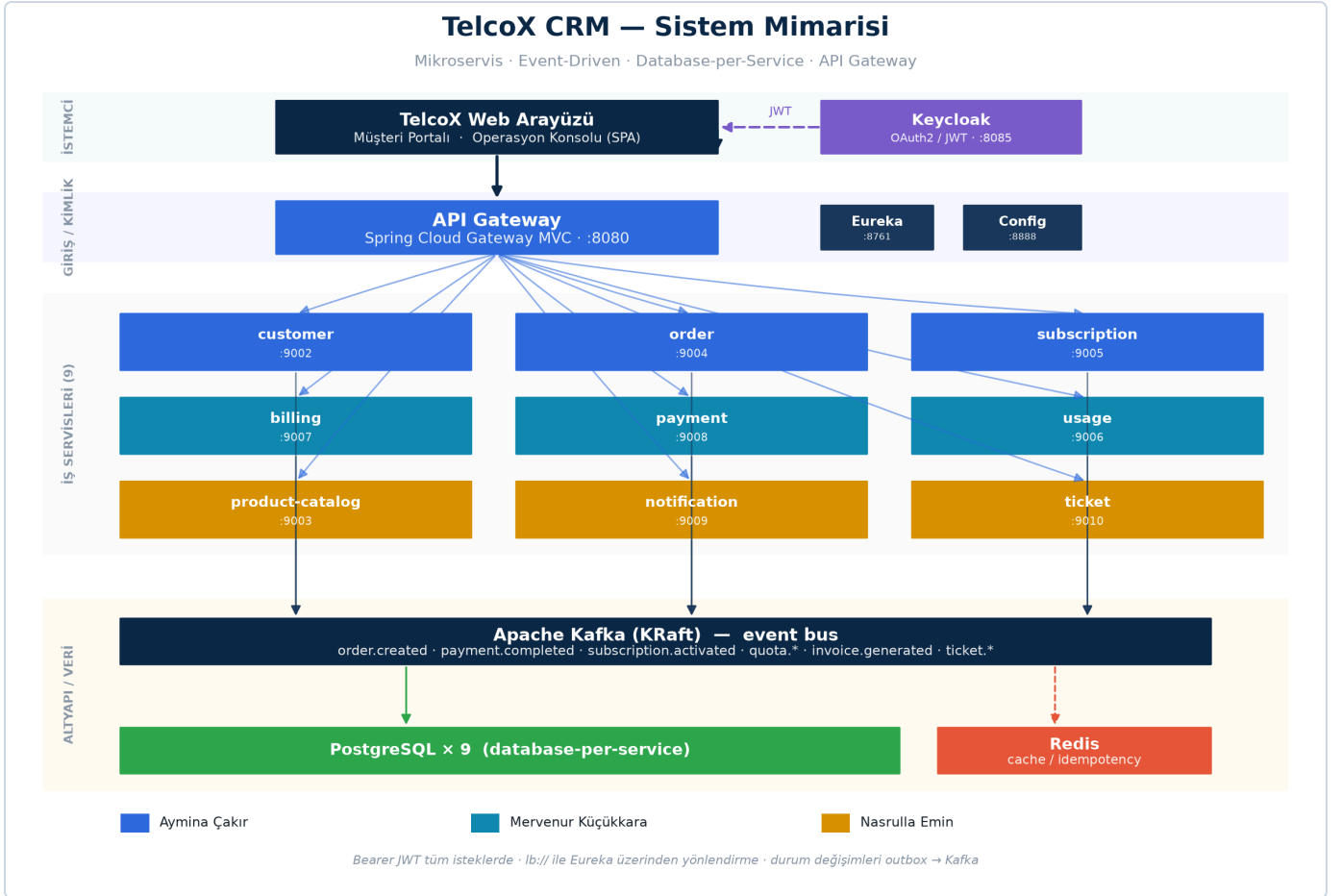
İlke	Ne demek	Bizde uygulaması
Database per service	Her servis yalnız kendi verisinin sahibi	9 ayrı PostgreSQL şeması; servisler arası FK yok, sadece ID referansı

İlke	Ne demek	Bizde uygulaması
Loose coupling	Servisler birbirine sıkı bağlı olmamalı	Asıl entegrasyon asenkron Kafka; senkron REST yalnız zorunlu yerde (Feign)
Bounded context (DDD)	Her modelin net bir geçerlilik sınırı	9 net domain: Müşteri, Katalog, Sipariş, Abonelik, Kullanım, Fatura, Ödeme, Bildirim, Talep
Atomicity	"DB yaz + event at" bölünememeli	Transactional Outbox: ikisi tek transaction'da
Idempotency	Aynı iş iki kez yapılıncı sonuç değişmemeli	processed_events + Idempotency-Key
Resilience	Bir servis çökse sistem ayakta kalmalı	Resilience4j circuit breaker/retry, outbox retry

Neden FK yerine ID referansı? Çünkü FK iki servisi aynı DB'ye zincirler ve bağımsız deploy/ölçeklemeyi öldürür. Tutarlılığı DB'ye değil, event'lere emanet ettik.

4. Sistem Mimarisi

Aşağıdaki diyagram, istemciden veri katmanına kadar tüm topolojiyi gösterir. Servisler sahiplerine göre renklendirilmiştir.



Şekil 1 — TelcoX CRM uçtan uca sistem mimarisi

Nasıl çalışır (özet)

- Kullanıcı tarayıcıdan **Keycloak'tan JWT** alır (ROPC). Web arayüzü gateway'in static/ klasöründen servis edilir → tek origin, CORS derdi yok.
- Her API isteği gateway'e gelir; gateway **JWT'yi doğrular**, lb://servis-adı ile Eureka üzerinden ilgili servise yönlendirir.
- Servisler durum değişimlerini **outbox üzerinden Kafka'ya** yayınlar; ilgili servisler asenkron tüketir (ödeme tamamlanınca abonelik aktive olur, bildirim gider).
- Config-server ortak ayarları Git'ten dağıtır; Redis sık okunan katalogu cache'ler; her servis kendi PostgreSQL şemasına yazar.

5. Teknoloji Yığını

Katman	Teknoloji	Neden
Dil / Framework	Java 21 · Spring Boot 3.4	Kurumsal standart; records, virtual-thread hazır
Servis topolojisi	Spring Cloud (Gateway MVC · Config · Eureka · OpenFeign)	Tek origin gateway, merkezi config, servis keşfi
Veri	PostgreSQL 17 (9 şema) · Flyway	Database-per-service; versiyonlu şema
Mesajlaşma	Apache Kafka (KRaft)	Asenkron, dayanıklı event bus — ZooKeeper'sız
Cache / idempotency	Redis 7	Cache-aside + idempotency store
Kimlik	Keycloak (OAuth2/JWT)	Endüstri standardı IdP; realm otomatik import
Dayanıklılık	Resilience4j	Circuit breaker, retry, bulkhead
Build	Maven (multi-module)	Tek repo, ortak parent pom
Container	Docker Compose · Kubernetes + HPA	Lokal orkestrasyon + prod ölçekleme
CI	GitHub Actions	Her push'ta build + test

6. Mikroservisler — Servis Servis Derinlik

Her geliştirici 3 servisin sahibidir; altyapı ortaktır.

Aymina Çakır — Müşteri Yolculuğunun Girişi

- customer-service (9002)** — Müşteri master kaydı, adres, KYC belgesi, KYC onay/red, iletişim. PII (kimlik) şifreleme.
- order-service (9004)** — Sipariş alımı ve Saga orchestration merkezi. order.created yayınlar, payment.completed/failed tüketir. Feign RequestInterceptor ile JWT taşır.

- **subscription-service (9005)** — Abonelik state machine, MSISDN havuzu + SIM tahsisi, MNP tablosu. payment.completed tüketip aboneliği aktive eder.

Mervenur Küçükkara — Para ve Kullanım

- **billing-service (9007)** — Bill-run scheduler, fatura üretimi, PDF çıktısı, kalemler, vergi. invoice.generated yayınlar; payment.completed ile PAID yapar.
- **payment-service (9008)** — Mock PSP, ödeme, retry, cüzdan, idempotency. order.created/invoice.generated tüketip ödeme oluşturur.
- **usage-service (9006)** — CDR tüketimi, dönemsel kota, %80/%100 eşik, aşım (overage). quota.threshold.reached, quota.exceeded, usage.aggregated yayınlar.

Nasrulla Emin — Katalog, İletişim, Destek

- **product-catalog-service (9003)** — Tarife/addon katalog, tarife versiyonlama, HYBRID tip + hedef segment, Redis cache.
- **notification-service (9009)** — Şablon tabanlı SMS/e-posta/push. Tüm event'leri tüketir, doğru şablonla bildirim üretir.
- **ticket-service (9010)** — Destek talebi, SLA takibi, önceliğe göre otomatik ekip atama, yorum akışı.

7. Kilit Mimari Desenler

7.1 Transactional Outbox — “event asla kaybolmaz”

Problem: “DB'ye yaz” ve “Kafka'ya at” iki ayrı sistem; biri başarılı diğeri başarısız olursa tutarsızlık doğar.

Çözüm: Event'i iş verisiyle aynı DB transaction'ında outbox_events tablosuna yaz; ayrı publisher Kafka'ya taşır. FAILED event retry ile denenir.

```
// subscription-service – abonelik kaydı + event tek transaction'da
Subscription saved = subscriptionRepository.save(subscription);
OutboxEvent outboxEvent = new OutboxEvent();
outboxEvent.setEventType("SubscriptionActivated");
outboxEvent.setPayload(objectMapper.writeValueAsString(activatedEvent));
outboxEvent.setStatus(OutboxStatus.PENDING);
outboxEventRepository.save(outboxEvent); // atomik: ya ikisi de, ya hicbiri
```

7.2 Saga (orchestration) — dağıtık işlem

Onboarding tek DB transaction'ı değil; 3 servise yayılır. Her adım lokal transaction yapar, sonucu event'le bildirir. Başarısızlıkta **kompansasyon** çalışır — ödeme başarısızsa abonelik SUSPENDED olur.

```
public void handlePaymentFailed(PaymentFailedEvent event) {
    if (processedEventRepository.existsById(event.eventId())) return; // idempotent
    subscriptionRepository.findById(event.orderId()).ifPresent(sub -> {
        if (sub.getStatus() == SubscriptionStatus.ACTIVE) {
            sub.setStatus(SubscriptionStatus.SUSPENDED); // telafi
            saveSubscriptionSuspendedOutboxEvent(sub);
        }
    });
}
```

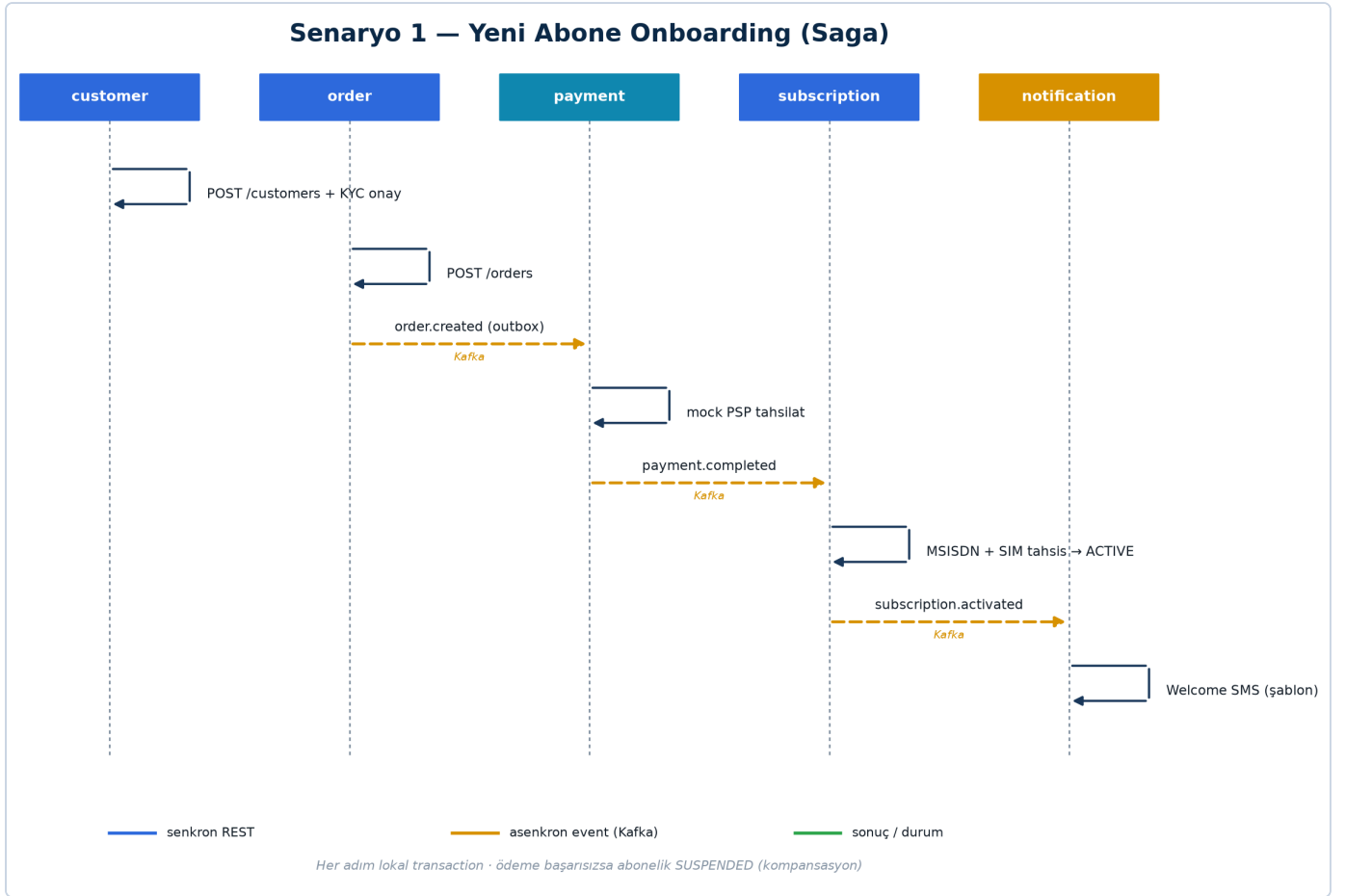

7.3 Idempotency & 7.4 Cache-aside / Pagination / Kota Eşiğı

- **Idempotency:** Kafka en-az-bir-kez teslim eder; her tüketici event id'sini processed_events'e yazar, tekrarı atlar.
- **Cache-aside:** product-catalog sık okunan tarifeyi Redis'te tutar; okuma yükünü DB'den alır.
- **Pagination:** Liste uçları PageResponse ile sayfalı (?page=0&size=20&sort=code,asc).
- **Kota eşiğı:** %80'de QuotaThresholdReached, %100'de QuotaExceeded; aşım UsageAggregated ile billing'e. Event customerId taşır → bildirim doğru kişiye.

8. Uçtan Uca İş Akışları (3 Senaryo)

Üç kabul senaryosunu da gerçek servisler üzerinde çalıştırıp doğruladık (bkz. Bölüm 11).

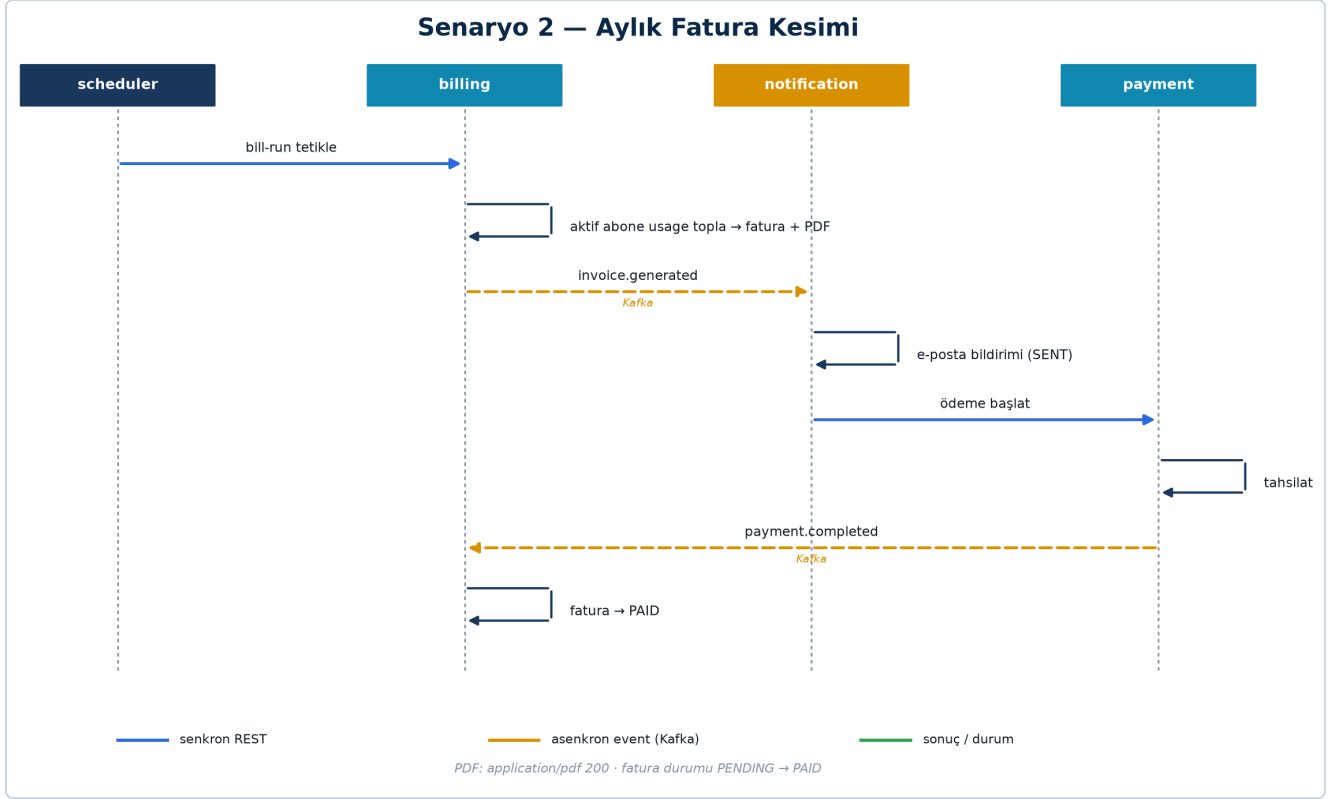
8.1 Yeni Abone Onboarding (Saga)



Şekil 2 — Onboarding: customer → order → payment → subscription → notification

8.2 Aylık Fatura Kesimi

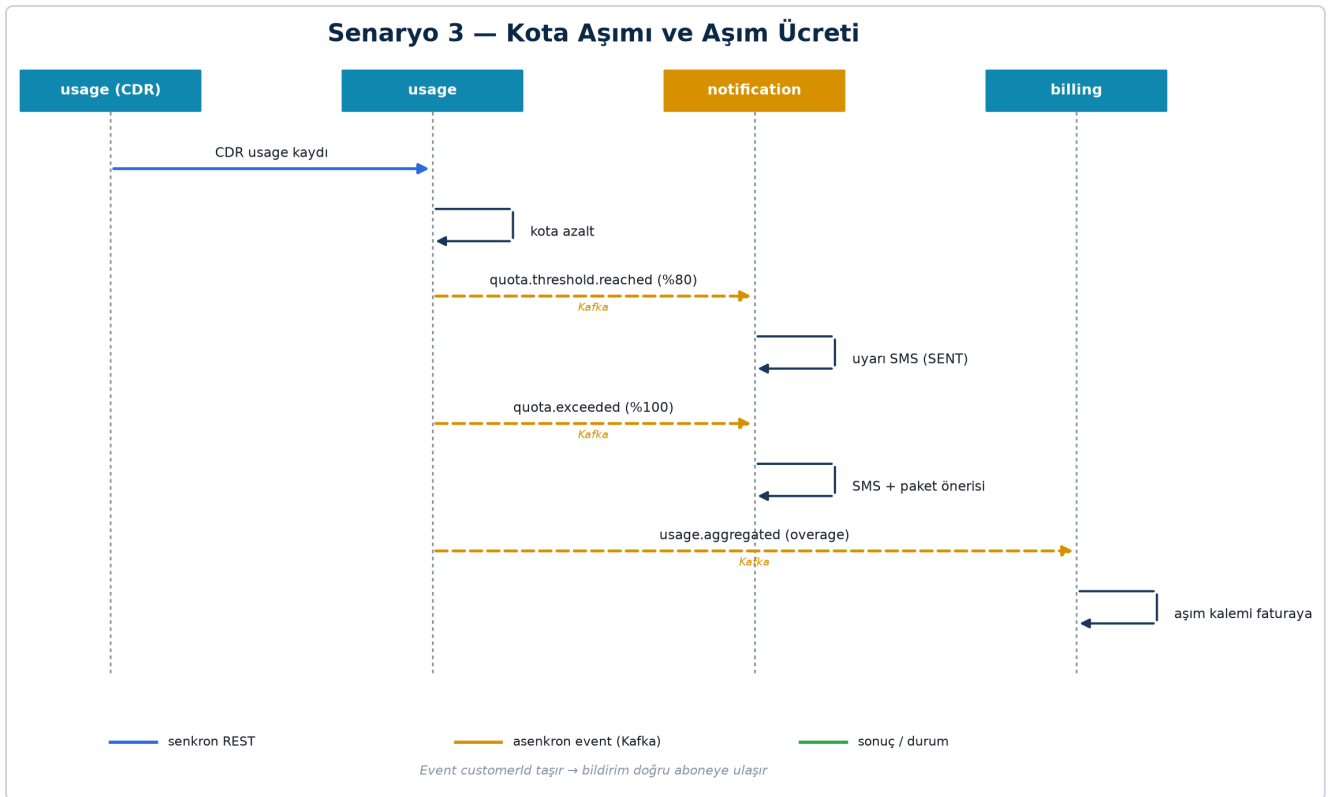
Senaryo 2 — Aylık Fatura Kesimi



Şekil 3 — Bill-run → fatura + PDF → bildirim → ödeme → PAID

8.3 Kota Aşımı ve Aşım Ücreti

Senaryo 3 — Kota Aşımı ve Aşım Ücreti



Şekil 4 — CDR → %80/%100 eşik SMS → overage faturaya

9. Güvenlik Mimarisi

- Keycloak realm telco-crm otomatik import. Kullanıcılar: **ops (ADMIN)**, **elif.aydin (USER)**, şifre telcox123.
- **Gateway + 9 servisin tamamı** OAuth2 resource-server: JWT imzası JWKS ile doğrulanır; realm rolleri ROLE_* authority'lerine map edilir.
- **Canlı RBAC**: @PreAuthorize("hasRole('ADMIN')") — tarife oluşturma, bill-run, KYC onayı yalnız ADMIN; USER admin uçlarında 403 alır.
- **Servisler arası**: Feign RequestInterceptor gelen Authorization başlığını taşır.
- **PII şifreleme**: müşteri kimlik verisi düz metin tutulmaz.

10. Web Arayüzü — TelcoX

Arayüz gateway'in içinden servis edilir (tek origin) ve **gerçek servislere bağlıdır** — mock değil; giriş gerçek JWT alır, tüm ekranlar canlı API verisi gösterir.

Müşteri Portalı (elif.aydin)

- Ana sayfa: gerçek tarife, güncel fatura, açık talep, kota çubukları.
- Paket Değiştir: gerçek katalogdan geçince abonelik + kota gerçekten güncellenir.
- Kullanımım: tarih aralığı CDR sorgusu. Faturalarım: ÖİV %10 + KDV %20 dökümü + PDF.
- Taleplerim: kategori + öncelik → gerçek POST /tickets. Profilim: kota %20 altına düşünce otomatik paket önerisi.

Operasyon Konsolu (ops)

- Dashboard / Müşteriler / Ürünler / Siparişler / Faturalama: gerçek API.
- Ticket Center: tüm müşterilerin talepleri (portaldan açılan dahil) + otomatik atanan ekip + İşleme AI / Çöz.
- Monitoring: gerçek servis sağlığı (/ops/health). **Alarm & Incident merkezi gerçek DOWN durumuna bağlıdır**: bir servis düşünce otomatik incident açılır, toparlanınca otomatik çözülür.

11. Kalite: Test, CI ve Kabul Kanıtları

3 kabul senaryosunu gerçek servisler üzerinde çalıştırdık — hepsi geçti:

#	Senaryo	Doğrulanmış çıktı
14.1	Onboarding	Abonelik ACTIVE, MSISDN atandı, welcome SMS SENT
14.2	Aylık Fatura	PDF 200/application-pdf, INVOICE_GENERATED e-posta SENT, fatura PAID
14.3	Kota Aşımı	%80 + %100 SMS SENT, overage → UsageAggregated billing'e



Şekil 5 — Teslim kapsamı ve doğrulama

- Birim testleri (JUnit + Mockito) her serviste; iş mantığı mock'larla doğrulanır.
- GitHub Actions CI: her push'ta mvn build + test.

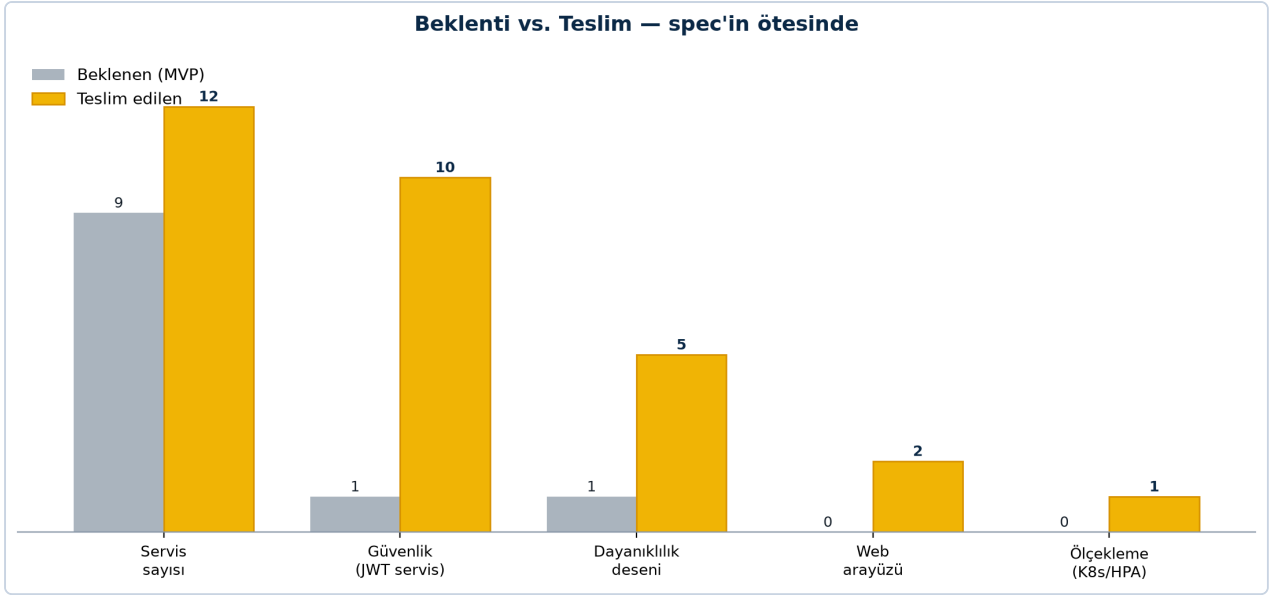
12. Karşılaştığımız Gerçek Problemler ve Çözümleri

Problem	Kök neden	Çözüm
Onboarding'de POST /orders 500	order_items.product_id NOT NULL; yeni akış productCode kullanıp productId'yi null bırakıyor	Flyway V3 ile kolon nullable
Gateway servislere ulaşamıyor (500)	Servisler Eureka'ya değişen makine IP'siyle kaydoluyordu	eureka.instance.hostname=localhost
Abonelik "0" görünüyor	Seed mnp_status='NONE' — enum'da yok, okurken 500	NOT_REQUESTED + kota customer_id + FREE SIM seed
Kota bildirimi kime gideceğini bilemiyor	QuotaThresholdEvent'te customerId yok	Event'e customerId eklendi (quota.getCustomerId())
Admin panelde talepler görünmüyor	Sadece sabit müşteri id'siyle sorgu	GET /tickets/all + gerçek-veri katmanı

13. DevOps ve Ölçeklenebilirlik

- Docker Compose: Kafka, 9x PostgreSQL, Redis, Keycloak — docker compose up -d.
- Tek komut başlatma: run-all.sh / stop-all.sh / status.sh / seed-demo-data.sh.
- CI: GitHub Actions build + test.
- Kubernetes: k8s/product-catalog/ — Deployment + HPA + Redis. HPA CPU'ya göre pod sayısını min-max aralığında otomatik ayarlar.

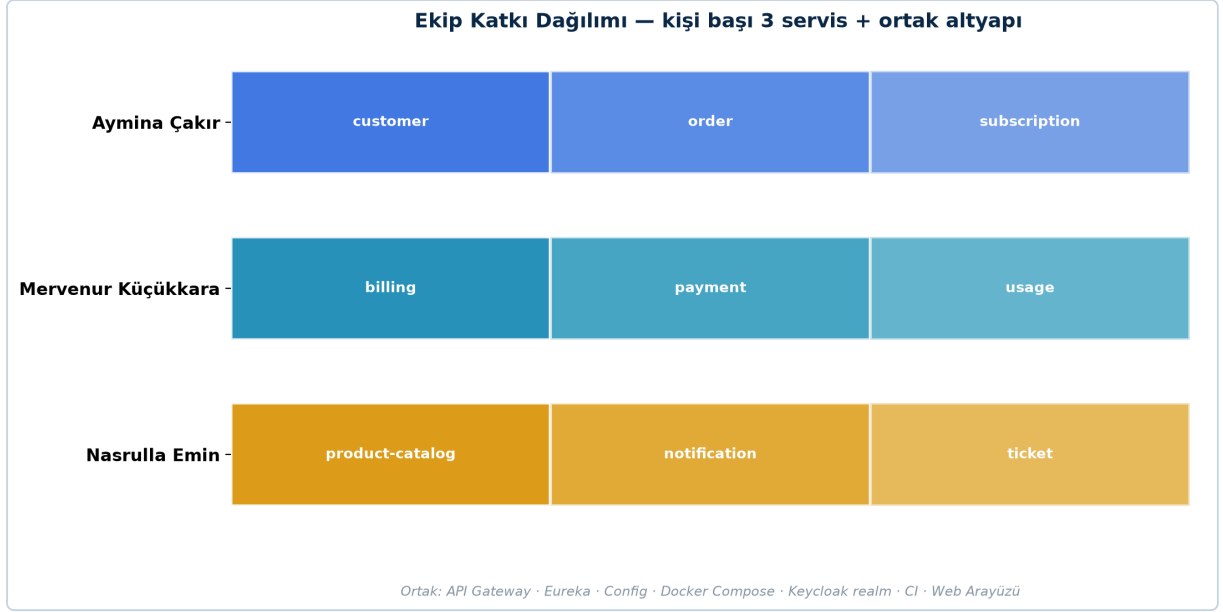
14. Beklenti vs. Teslim



Şekil 6 — Beklenen MVP ve teslim edilen kapsam

Beklenen (MVP)	Teslim	Spec üstü eklediklerimiz
9 servis + gateway/eureka/config	✓	—
Database-per-service + Kafka	✓	Outbox + FAILED retry
Saga (onboarding)	✓	Kompansasyon (askıya alma)
JWT güvenlik	✓	9 servisin tamamı + canlı RBAC + Feign relay
3 kabul senaryosu	✓	Uçtan uca test edildi + otomatik seed
—	+	Gerçek web arayüzü (portal + ops konsolu)
—	+	Redis cache · pagination · PDF fatura · PII şifreleme
—	+	Resilience4j · idempotency · K8s + HPA · CI

15. Ekip ve Katkıları



Şekil 7 — Ekip katkı dağılımı

Geliştirici	Servisler	Öne çıkan katkılar
Aymina Çakır	customer · order · subscription	KYC + PII şifreleme; Saga orchestration; MSISDN/SIM lifecycle; Feign token interceptor; productCode migration
Mervenur Küçük kara	billing · payment · usage	Bill-run + PDF; mock PSP + retry + idempotency + cüzdan + Flyway; kota/CDR + aşım bildirimleri
Nasrulla Emin	product-catalog · notification · ticket	Tarife versiyonlama + Redis cache; şablon bildirim; SLA + otomatik ekip atama; ops konsolu gerçek-veri + Alarm/Incident

Ortak çalışma: API Gateway, Eureka, config-server, Docker Compose, GitHub Actions CI, Keycloak realm, TelcoX web arayüzü, kabul testleri, dokümantasyon.

16. Sonuç ve Gelecek Çalışma

Öğrendiklerimiz

- Dağıtık transaction'ın gerçek zorluğu ve outbox + saga ile pratikte çözümü.
- Event-driven tasarımın gevşek bağ avantajı — bir servis yavaşsa diğerleri beklemez.
- JWT/RBAC ile uçtan uca güvenliğin gateway + servis katmanında kurulması.
- Servis keşfi (Eureka), merkezi config ve 12-factor yaklaşımının değeri.

Gelecek çalışma

- Prometheus + Grafana ile gerçek metrik/CPU/pod.
- Zipkin/OpenTelemetry ile dağıtık trace.
- Tüm servisler için tam Kubernetes manifestleri + service mesh.
- Contract testing ile event uyumluluğu.

17. Hızlı Referans

```
# Servisler & portlar
gateway :8080   eureka :8761   config :8888   keycloak :8085
customer :9002  product-catalog :9003  order :9004  subscription :9005
usage :9006     billing :9007  payment :9008  notification :9009  ticket :9010

# Çalıştırma
cd docker && docker compose up -d && cd ..   # altyapı
./scripts/run-all.sh                        # 12 servis (status.sh -> 12/12)
./scripts/seed-demo-data.sh                 # demo veri
# http://localhost:8080/TelcoX.html   .   ops / elif.aydin . sifre telcox123
```

TelcoX CRM — Turkcell Geleceği Yazarlar Bootcamp 2026

Aymina Çakır · Mervenur Küçükkara · Nasrulla Emin